

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Conexión

Conceptos fundamentales que dominamos

- **'curve_fit'**: Ajustar funciones arbitrarias (exponenciales, potencias, etc.)
- **Matriz de covarianza**: Extracción automática de incertidumbres con 'pcov'
- **Ponderación con 'sigma'**: Dar mayor peso a mediciones precisas
- **Ejemplo físico**: Relación masa-luminosidad (log-log vs ajuste directo)

Objetivo de hoy (P2)

Consolidar 'curve_fit' aplicándolo a **4 problemas físicos reales** con diferentes modelos matemáticos.

- **Consolidar** el uso de 'curve_fit' en contextos variados
- **Aplicar** modelos exponenciales, oscilatorios y saturación
- **Desarrollar** habilidades para extraer e interpretar incertidumbres
- **Integrar** visualización con barras de error y bandas de confianza

Ejercicios Prácticos Integrados

- **Enfoque 100% práctico:** Resolver 4 problemas con 'curve_fit'
- **Integración de conceptos:** Combinar ajuste + incertidumbres + visualización
- **Contextos físicos:** Decaimiento radiactivo, oscilaciones, crecimiento de poblaciones, ley de enfriamiento

Meta de la Sesión

Lograr que cada estudiante se sienta cómodo eligiendo y ajustando el modelo físico apropiado para sus datos.



Decaimiento Radiactivo de C-14

Enunciado

- Modelo: $N(t) = N_0 e^{-\lambda t}$ donde $\lambda = \ln(2)/T_{1/2}$
- Datos simulados: 15 mediciones de actividad (N) vs tiempo (t) en años
- Valores reales: $N_0 = 1000$ Bq, $T_{1/2} = 5730$ años
- Ruido Gaussiano: $\sigma = 5\%$ de la señal

• Tareas:

1. Defina función 'exponencial_decay(t, N_0, λ)'
1. Use 'curve_fit' para ajustar los parámetros
2. Extraiga incertidumbres desde 'pcov'
3. Grafique datos con barras de error + curva ajustada
4. Calcule $T_{1/2}$ ajustado y compare con el real

Conceptos: Decaimiento exponencial, vida media

Física: Datación por radiocarbono



Enunciado

- Modelo: $x(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$
- Parámetros reales: $A = 10 \text{ cm}$, $\gamma = 0.1 \text{ s}^{-1}$, $\omega = 2\pi \text{ rad/s}$, $\phi = 0$
- Genere 50 puntos en $t \in [0, 20] \text{ s}$ con ruido $\sigma = 0.3 \text{ cm}$
- **Tareas:**
 1. Defina función `'oscilador_amortiguado(t, A, gamma, omega, phi)'`
 1. Ajuste con `'curve_fit'` usando `'p0=[8, 0.05, 6, 0]'` (valores iniciales)
 2. Calcule periodo $T = 2\pi/\omega$ y tiempo de decaimiento $\tau = 1/\gamma$
 3. Grafique con banda de incertidumbre (use `'fill_between'`)
 3. Reporte parámetros con formato: $A = X \pm \sigma_A$

Conceptos: Oscilaciones amortiguadas, valores iniciales

Aplicación: Análisis de sistemas mecánicos disipativos



Enunciado

- Modelo logístico: $N(t) = \frac{K}{1+e^{-r(t-t_0)}}$
- Contexto: Formación estelar en un cúmulo con capacidad limitada
- Parámetros reales: $K = 500$ estrellas, $r = 0.3 \text{ Myr}^{-1}$, $t_0 = 5 \text{ Myr}$
- Genere 25 mediciones en $t \in [0, 20] \text{ Myr}$ con ruido $\sigma = 10$ estrellas
- Tareas:
 1. Defina función 'logistica(t, K, r, t0)'
 2. Ajuste con 'curve_fit' (use 'p0=[400, 0.2, 4]')
 3. Extraiga incertidumbres y calcule intervalos del 68% de confianza
 4. Grafique datos + curva ajustada + zona de confianza
 5. Interprete: ¿A qué tiempo se alcanza el 90% de la capacidad?

Conceptos: Crecimiento saturado, asíntotas

Física: Formación estelar en entornos densos



Enfriamiento de Enana Blanca

Enunciado

- Modelo: $T(t) = T_{\text{amb}} + (T_0 - T_{\text{amb}})e^{-kt}$
- Parámetros reales: $T_0 = 25000$ K, $T_{\text{amb}} = 2.7$ K (CMB), $k = 0.05$ Gyr $^{-1}$
- Genere 20 mediciones en $t \in [0, 50]$ Gyr con ruido $\sigma = 200$ K
- Tareas:
 1. Defina función 'enfriamiento($t, T_{\text{amb}}, T_0, k$)'
 1. Ajuste con 'curve_fit' y extraiga incertidumbres
 2. Calcule tiempo de semi-enfriamiento: $t_{1/2} = \frac{\ln 2}{k}$
 3. Grafique en escala semi-log (`plt.semilogy`)
 4. Compare ajuste con valores reales usando χ^2_{red}

Conceptos: Enfriamiento exponencial, escalas temporales largas

Aplicación: Evolución térmica de objetos compactos

Soluciones de Referencia

Estructura común de todas las soluciones

1. Definir función del modelo físico
2. Generar datos simulados con ruido realista
3. Ajustar con 'curve_fit' y proporcionar 'p0' si es necesario
4. Extraer parámetros óptimos y sus incertidumbres
5. Visualizar datos + ajuste + bandas de confianza
6. Calcular magnitudes físicas derivadas

Estas soluciones son plantillas reutilizables para análisis de datos reales.



Decaimiento Radiactivo

```

1 from scipy.optimize import curve_fit
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Modelo de decaimiento exponencial
6 def exponencial_decay(t, N0, lambda_val):
7     return N0 * np.exp(-lambda_val * t)
8
9 # Parámetros reales
10 T_half_real = 5730 # años
11 lambda_real = np.log(2) / T_half_real
12 N0_real = 1000 # Bq
13
14 # Generar datos
15 np.random.seed(42)
16 t = np.linspace(0, 15000, 15)
17 N_real = exponencial_decay(t, N0_real, lambda_real)
18 sigma_N = 0.05 * N_real # 5% de ruido
19 N_obs = N_real + np.random.normal(0, sigma_N)

```

```

18 # Ajustar con curve_fit
19 popt, pcov = curve_fit(
20     exponencial_decay, t, N_obs,
21     sigma=sigma_N, absolute_sigma=True
22 )
23 N0_fit, lambda_fit = popt
24 perr = np.sqrt(np.diag(pcov))
25
26 # Calcular T_1/2 ajustado
27 T_half_fit = np.log(2) / lambda_fit
28 sigma_T_half = T_half_fit * (perr[1]/lambda_fit)
29
30 print(f"N0 = {N0_fit:.1f} ± {perr[0]:.1f} Bq")
31 print(f"T_1/2 = {T_half_fit:.0f} ±
↪ {sigma_T_half:.0f} años")
32 print(f"Real: {T_half_real} años")
33 print(f"Error relativo:
↪ {abs(T_half_fit-T_half_real)/T_half_real*100:.1f}%")

```



Visualización

```

1 # Graficar resultados
2 t_fit = np.linspace(0, 15000, 200)
3 N_fit = exponencial_decay(t_fit, *popt)
4
5 plt.figure(figsize=(10, 6))
6 plt.errorbar(t, N_obs, yerr=sigma_N, fmt='o', capsize=3,
7             label='Mediciones', alpha=0.7)
8 plt.plot(t_fit, N_fit, 'r-', lw=2,
9          label=f'Ajuste:  $T_{1/2} = \{T\_half\_fit:.0f\} \pm \{\sigma\_T\_half:.0f\}$  años')
10 plt.axhline(N0_real/2, color='gray', linestyle='--', alpha=0.5,
11            label=f' $N_0/2$  ( $t = \{T\_half\_real\}$  años)')
12 plt.xlabel('Tiempo (años)')
13 plt.ylabel('Actividad N (Bq)')
14 plt.title('Decaimiento Radiactivo de C-14')
15 plt.legend()
16 plt.grid(alpha=0.3)
17 plt.show()

```

Discusión: La incertidumbre en λ se propaga a $T_{1/2}$ mediante $\sigma_{T_{1/2}} = T_{1/2} \cdot (\sigma_{\lambda}/\lambda)$.



Oscilador Amortiguado

```

1  # Modelo oscilador amortiguado
2  def oscilador(t, A, gamma, omega, phi):
3      return A * np.exp(-gamma*t) * np.cos(omega*t +
4          ↪ phi)
5
6  # Parámetros reales
7  A_real, gamma_real = 10, 0.1 # cm, s-1
8  omega_real, phi_real = 2*np.pi, 0 # rad/s
9
10 # Generar datos
11 np.random.seed(42)
12 t = np.linspace(0, 20, 50)
13 x_real = oscilador(t, A_real, gamma_real,
14     ↪ omega_real, phi_real)
15
16 sigma_x = np.full_like(x_real, 0.3) # 0.3 cm
17 x_obs = x_real + np.random.normal(0, sigma_x)
18
19 # Ajustar (necesitamos valores iniciales)
20 p0 = [8, 0.05, 6, 0] # estimaciones
21 popt, pcov = curve_fit(
22     oscilador, t, x_obs, p0=p0,
23     sigma=sigma_x, absolute_sigma=True
24 )

```

```

20 A_fit, gamma_fit, omega_fit, phi_fit = popt
21 perr = np.sqrt(np.diag(pcov))
22
23 # Magnitudes derivadas
24 T_periodo = 2*np.pi / omega_fit
25 tau_decay = 1 / gamma_fit
26
27 print(f"Amplitud A = {A_fit:.2f} ± {perr[0]:.2f}
28     ↪ cm")
29 print(f"Amortiguamiento γ = {gamma_fit:.3f} ±
30     ↪ {perr[1]:.3f} s-1")
31 print(f"Frecuencia ω = {omega_fit:.2f} ±
32     ↪ {perr[2]:.2f} rad/s")
33 print(f"Fase φ = {phi_fit:.2f} ± {perr[3]:.2f}
34     ↪ rad")
35 print(f"\nPeriodo T = {T_periodo:.2f} s")
36 print(f"Tiempo de decaimiento τ = {tau_decay:.1f}
37     ↪ s")

```




Banda de Confianza

```
1 # Graficar con banda de confianza
2 t_fit = np.linspace(0, 20, 300)
3 x_fit = oscilador(t_fit, *popt)
4
5 # Banda de confianza (simplificada:  $\pm 1$  sigma en amplitud)
6 x_upper = oscilador(t_fit, A_fit+perr[0], gamma_fit, omega_fit, phi_fit)
7 x_lower = oscilador(t_fit, A_fit-perr[0], gamma_fit, omega_fit, phi_fit)
8
9 plt.figure(figsize=(12, 6))
10 plt.errorbar(t, x_obs, yerr=sigma_x, fmt='o', capsize=2,
11             label='Mediciones', alpha=0.6)
12 plt.plot(t_fit, x_fit, 'r-', lw=2, label='Ajuste')
13 plt.fill_between(t_fit, x_lower, x_upper, color='red', alpha=0.2,
14                 label='Banda  $\pm 1\sigma$ ')
15 plt.xlabel('Tiempo (s)')
16 plt.ylabel('Posición x (cm)')
17 plt.title('Oscilador Amortiguado')
18 plt.legend()
19 plt.grid(alpha=0.3)
20 plt.show()
```

Nota: Para banda completa, propagar todas las incertidumbres con Monte Carlo.

Soluciones de Referencia $\ni$ Solución 3:

Crecimiento Logístico



```
1 # Modelo logístico
2 def logistica(t, K, r, t0):
3     return K / (1 + np.exp(-r*(t - t0)))
4
5 # Parámetros reales
6 K_real, r_real, t0_real = 500, 0.3, 5 # estrellas,
   ↪ Myr-1, Myr
7
8 # Generar datos
9 np.random.seed(42)
10 t = np.linspace(0, 20, 25)
11 N_real = logistica(t, K_real, r_real, t0_real)
12 sigma_N = np.full_like(N_real, 10) # 10 estrellas
13 N_obs = N_real + np.random.normal(0, sigma_N)
14
15 # Ajustar
16 p0 = [400, 0.2, 4]
17 popt, pcov = curve_fit(
18     logistica, t, N_obs, p0=p0,
19     sigma=sigma_N, absolute_sigma=True
20 )
```

```
18 K_fit, r_fit, t0_fit = popt
19 perr = np.sqrt(np.diag(pcov))
20
21 print(f"Capacidad K = {K_fit:.0f} ± {perr[0]:.0f}
   ↪ estrellas")
22 print(f"Tasa r = {r_fit:.3f} ± {perr[1]:.3f}
   ↪ Myr-1")
23 print(f"Punto medio t0 = {t0_fit:.1f} ±
   ↪ {perr[2]:.1f} Myr")
24
25 # Tiempo al 90% de K
26 def t_90(K, r, t0):
27     return t0 + np.log(9) / r # de 0.9K =
   ↪ K/(1+exp(-r(t-t0)))
28
29 t_90_val = t_90(K_fit, r_fit, t0_fit)
30 print(f"\nTiempo al 90% de K: {t_90_val:.1f} Myr")
```



Visualización

```
1 # Graficar
2 t_fit = np.linspace(0, 20, 200)
3 N_fit = logistica(t_fit, *popt)
4
5 plt.figure(figsize=(10, 6))
6 plt.errorbar(t, N_obs, yerr=sigma_N, fmt='o', capsize=3,
7             label='Observaciones', alpha=0.7)
8 plt.plot(t_fit, N_fit, 'r-', lw=2, label='Ajuste logístico')
9 plt.axhline(K_fit, color='gray', linestyle='--', alpha=0.5, label=f'K={K_fit:.0f}')
10 plt.axhline(0.9*K_fit, color='orange', linestyle=':', alpha=0.5, label='90% de K')
11 plt.axvline(t_90_val, color='orange', linestyle=':', alpha=0.5)
12 plt.xlabel('Tiempo (Myr)')
13 plt.ylabel('Número de Estrellas N')
14 plt.title('Formación Estelar en Cúmulo')
15 plt.legend()
16 plt.grid(alpha=0.3)
17 plt.show()
```

Interpretación: El modelo captura bien la saturación de la formación estelar debido a agotamiento de gas.



Ley de Enfriamiento

```

1 # Modelo de enfriamiento
2 def enfriamiento(t, T_amb, T0, k):
3     return T_amb + (T0 - T_amb) * np.exp(-k*t)
4
5 # Parámetros reales
6 T0_real, T_amb_real = 25000, 2.7 # K
7 k_real = 0.05 # Gyr-1
8
9 # Generar datos
10 np.random.seed(42)
11 t = np.linspace(0, 50, 20)
12 T_real = enfriamiento(t, T_amb_real, T0_real,
    ↪ k_real)
13 sigma_T = np.full_like(T_real, 200) # 200 K
14 T_obs = T_real + np.random.normal(0, sigma_T)
15
16 # Ajustar
17 popt, pcov = curve_fit(
18     enfriamiento, t, T_obs,
19     sigma=sigma_T, absolute_sigma=True
20 )

```

```

18 T_amb_fit, T0_fit, k_fit = popt
19 perr = np.sqrt(np.diag(pcov))
20
21 # Tiempo de semi-enfriamiento
22 t_half = np.log(2) / k_fit
23 sigma_t_half = t_half * (perr[2]/k_fit)
24
25 print(f"T_amb = {T_amb_fit:.1f} ± {perr[0]:.1f} K")
26 print(f"T0 = {T0_fit:.0f} ± {perr[1]:.0f} K")
27 print(f"k = {k_fit:.4f} ± {perr[2]:.4f} Gyr-1")
28 print(f"\nt1/2 = {t_half:.1f} ± {sigma_t_half:.1f}
    ↪ Gyr")
29
30 # Calcular chi2 reducido
31 residuos = T_obs - enfriamiento(t, *popt)
32 chi2_red = np.sum((residuos/sigma_T)**2) / (len(t)
    ↪ - 3)
33 print(f"χ2_red = {chi2_red:.2f}")

```



Gráfico Semi-Log

```
1 # Graficar en escala semi-log
2 t_fit = np.linspace(0, 50, 200)
3 T_fit = enfriamiento(t_fit, *popt)
4
5 plt.figure(figsize=(10, 6))
6 plt.errorbar(t, T_obs, yerr=sigma_T, fmt='o', capsize=3,
7             label='Mediciones', alpha=0.7)
8 plt.plot(t_fit, T_fit, 'r-', lw=2,
9          label=f'Ajuste:  $t_{1/2} = \{t\_half:.1f\} \pm \{\sigma\_t\_half:.1f\}$  Gyr')
10 plt.axhline(T_amb_real, color='blue', linestyle='--', alpha=0.5,
11            label=f'T_CMB =  $\{T\_amb\_real\}$  K')
12 plt.yscale('log')
13 plt.xlabel('Tiempo (Gyr)')
14 plt.ylabel('Temperatura (K) [escala log]')
15 plt.title('Enfriamiento de Enana Blanca')
16 plt.legend()
17 plt.grid(alpha=0.3, which='both')
18 plt.show()
```

Nota: $\chi^2_{\text{red}} \approx 1$ indica buen ajuste; > 2 sugiere modelo inadecuado o errores subestimados.

Estructuras y conceptos integrados

Todos los ejercicios comparten:

- Definición de función del modelo físico
- Generación de datos simulados con ruido Gaussiano
- Uso de 'curve_fit' con ponderación por 'sigma'
- Extracción de incertidumbres desde 'pcov'
- Cálculo de magnitudes físicas derivadas
- Visualización con barras de error

Diferencias clave

- **Ejercicio 2 y 3:** Requieren valores iniciales 'p0' (modelos no lineales complejos)
- **Ejercicio 4:** Uso de escala logarítmica para visualizar decaimientos largos
- **Ejercicio 1:** Propagación de errores explícita ($T_{1/2}$ desde λ)

Consolidación y Retroalimentación

Reflexión sobre los ejercicios

- ¿Qué modelo resultó más difícil de ajustar? ¿Por qué?
- ¿Encontraron problemas con valores iniciales en oscilador/logística?
- ¿Cómo interpretaron las incertidumbres en el contexto físico?
- ¿Lograron calcular magnitudes derivadas ($T_{1/2}$, *periodo*, etc.)?

Objetivo

Desarrollar intuición para elegir modelos apropiados y validar ajustes con métricas como χ^2_{red} .

Errores frecuentes

- **Olvidar valores iniciales 'p0'**: Modelos no lineales necesitan buenas estimaciones
- **No usar 'absolute_sigma=True'**: Las incertidumbres quedan en escala relativa
- **Extrapolar fuera del rango de datos**: 'curve_fit' ajusta, no predice
- **Ignorar (χ^2_{red})**: Un valor lejano de 1 indica problemas
- **No propagar errores correctamente**: Usar fórmulas de propagación para magnitudes derivadas

Consejo

Siempre visualiza residuos: 'plt.plot(t, T_obs - T_fit)' revela patrones sistemáticos.

Conclusiones

Técnicas consolidadas hoy

Ajuste con 'curve_fit':

- Decaimiento exponencial (radiactividad)
- Oscilaciones amortiguadas (sistemas disipativos)
- Crecimiento saturado (poblaciones)
- Enfriamiento (ley de Newton)

Habilidades desarrolladas:

- Extracción e interpretación de incertidumbres
- Propagación de errores a magnitudes derivadas
- Visualización con bandas de confianza
- Validación con (χ^2_{red})

Habilidad adquirida

Ajustar modelos físicos arbitrarios a datos reales con evaluación rigurosa de incertidumbres.

Pasos estándar para análisis con 'curve_fit'

1. **Definir** función del modelo físico
2. **Estimar** valores iniciales 'p0' (si es necesario)
3. **Ajustar** con 'curve_fit' y 'sigma'
4. **Extraer** parámetros y 'pcov'
5. **Calcular** incertidumbres con 'np.sqrt(np.diag(pcov))'
6. **Derivar** magnitudes físicas con propagación de errores
7. **Validar** con (χ^2_{red}) y gráficos de residuos
8. **Visualizar** datos + ajuste + banda de confianza

Este workflow es aplicable a cualquier problema de ajuste en física/astronomía.

- **Próxima semana:** Técnicas avanzadas (interpolación, bootstrapping, selección de modelos)
- **Recomendación de estudio:**
 - Practicar con datasets propios (Kaggle, repositorios astronómicos)
 - Explorar 'lmfit' (alternativa a 'curve_fit' con más funcionalidades)
- **Práctica sugerida:**
 - Aplicar 'curve_fit' a curvas de luz reales (TESS, Kepler)
 - Experimentar con modelos físicos de su área de interés

¡Excelente trabajo consolidando 'curve_fit'!

Apéndice: Ver también

Apéndice: Ver también $\ni$ Extra: Propagación de Errores con Covarianza Completa

```
1 # Ejemplo: Si  $z = f(a, b)$  con incertidumbres correlacionadas
2 # Usar fórmula general de propagación:
3 #  $\sigma_z^2 = (\partial f / \partial a)^2 \sigma_a^2 + (\partial f / \partial b)^2 \sigma_b^2 + 2(\partial f / \partial a)(\partial f / \partial b)\text{cov}(a, b)$ 
4
5 # Para  $T_{1/2} = \ln(2) / \lambda$ :
6 def prop_error_T_half(lambda_fit, pcov_lambda):
7     deriv = -np.log(2) / lambda_fit**2
8     var_T = deriv**2 * pcov_lambda
9     return np.sqrt(var_T)
10
11 # pcov[1,1] es la varianza de lambda
12 sigma_T_half = prop_error_T_half(lambda_fit, pcov[1,1])
```

Nota: Para funciones de múltiples variables, usar la matriz completa 'pcov'.

```
1 # Si no sabes qué p0 usar, prueba múltiples:
2 from scipy.optimize import curve_fit
3
4 p0_candidates = [
5     [8, 0.05, 6, 0], # conservador
6     [12, 0.15, 7, 0.5], # agresivo
7     [10, 0.1, 2*np.pi, 0] # basado en física
8 ]
9
10 results = []
11 for p0 in p0_candidates:
12     try:
13         popt, pcov = curve_fit(oscilador, t, x_obs, p0=p0, sigma=sigma_x)
14         chi2_red = calcular_chi2_red(t, x_obs, sigma_x, popt)
15         results.append((chi2_red, popt))
16     except RuntimeError:
17         continue
18
19 # Elegir el de menor chi2_red
20 best = min(results, key=lambda x: x[0])
21 print(f"Mejor ajuste: chi2_red={best[0]:.2f}")
```



```
1 # Criterio de Información de Akaike (introducción simple)
2 def calcular_AIC(residuos, sigma, n_params):
3     chi2 = np.sum((residuos/sigma)**2)
4     log_likelihood = -0.5 * chi2
5     AIC = 2*n_params - 2*log_likelihood
6     return AIC
7
8 # Ejemplo: comparar modelo lineal vs cuadrático
9 residuos_lin = y_obs - modelo_lineal(x, *popt_lin)
10 residuos_quad = y_obs - modelo_cuadratico(x, *popt_quad)
11
12 AIC_lin = calcular_AIC(residuos_lin, sigma, 2) # 2 parámetros
13 AIC_quad = calcular_AIC(residuos_quad, sigma, 3) # 3 parámetros
14
15 print(f"AIC lineal: {AIC_lin:.1f}")
16 print(f"AIC cuadrático: {AIC_quad:.1f}")
17 print("Mejor modelo:", "lineal" if AIC_lin < AIC_quad else "cuadrático")
```

Nota: AIC menor indica mejor balance entre ajuste y complejidad.